

Event Handling in Java

Event handling is a fundamental concept in Java for creating interactive GUI applications. It is the process of responding to user actions, such as clicking a button, moving the mouse, or typing on a keyboard, to trigger specific actions or responses in the program.

What is Event Handling?

- **Definition:** Event handling is the mechanism by which an application responds to user interactions with its graphical interface components, such as buttons, text fields, and menus.
- **Purpose:** It enables programs to detect and process user actions, providing interactivity and functionality in GUI-based applications.
- **Examples:**
 - Clicking a "Submit" button in a form.
 - Typing text in a `JTextField`.
 - Moving or dragging a mouse pointer across the screen.

Importance in GUI Applications

1. **Interactivity:** Event handling allows users to interact with the application by performing actions like clicks, drags, and keystrokes.
2. **Dynamic Behavior:** Enables applications to perform specific tasks dynamically, such as opening a new window, updating data, or responding to user input.
3. **Enhanced User Experience:** Makes applications more user-friendly by providing immediate feedback to user actions.
4. **Modularity:** Separates the logic for handling user actions (event listeners) from the main application, making the code easier to maintain.

Core Concepts

1. Event

An event represents an occurrence or action that the program can respond to.

- **Definition:** A user's interaction with a GUI component, such as clicking a button or moving the mouse, triggers an event.
- **Examples of Events:**
 - `ActionEvent`: Triggered by a button click.
 - `MouseEvent`: Triggered by mouse movements or clicks.
 - `KeyEvent`: Triggered by keystrokes.

2. Event Source

The event source is the object that generates the event.

- **Definition:** It is the GUI component that detects the user's action and notifies the program of the event.
- **Examples:**
 - A `JButton` generates an `ActionEvent` when clicked.
 - A `JTextField` generates an `ActionEvent` when the Enter key is pressed.
- **Role:** The event source is responsible for:
 - Detecting user interactions.
 - Notifying the registered event listeners about the event.

3. Event Listener

The event listener is the object that handles the event.

- **Definition:** It is an object that implements one or more event listener interfaces (e.g., `ActionListener`, `MouseListener`) and contains methods that define how to respond to specific events.
- **Role:**
 - Listens to events generated by the event source.
 - Responds by executing the code within the implemented methods.
- **How it Works:**
 - The event listener is registered with the event source.
 - When an event occurs, the event source notifies the listener by calling the appropriate method.

4. Event Object

The event object encapsulates all the information about the event.

- **Definition:** A specialized class that contains details about the event, such as its type, source, and any associated data.
- **Common Event Classes:**
 - `ActionEvent`: Contains information about an action event, such as a button click.
 - `MouseEvent`: Provides details about mouse events, like coordinates of the click.
 - `KeyEvent`: Includes information about key presses, such as the key code.
- **Methods in Event Objects:**
 - `getSource()`: Returns the source of the event.
 - `getActionCommand()`: Retrieves the action command string (used in `ActionEvent`).
 - `getWhen()`: Returns the time when the event occurred.

Example: Basic Event Handling

Here's a simple example of event handling using a button click:

```
import javax.swing.*;
import java.awt.event.*;

public class EventHandlingExample {
    public static void main(String[] args) {
        // Create a JFrame
        JFrame frame = new JFrame("Event Handling Example");
        frame.setSize(300, 200);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        // Create a JButton
        JButton button = new JButton("Click Me!");
        frame.add(button);

        // Add ActionListener to the button
        button.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                // Handle the event
                System.out.println("Button clicked!");
            }
        });

        // Make the frame visible
        frame.setVisible(true);
    }
}
```

Key Points to Emphasize

- The distinction between **event source** and **event listener**.
- The flow of event handling:
 1. **Event occurs** → 2. **Event is detected by the source** → 3. **Source notifies the listener** → 4. **Listener handles the event**.
- Practical examples for clarity and hands-on experience.

java.awt package

The `java.awt` package (Abstract Window Toolkit) is part of the Java Standard Library and provides classes for creating graphical user interface (GUI) components, handling events, and working with graphics, images, and colors. It's one of the oldest GUI toolkits in Java and serves as the foundation for Swing.

1. Purpose of java.awt

- Provides tools to create and manage graphical user interfaces.
- Supports event handling for user interactions.
- Offers classes for 2D drawing, image manipulation, and layout management.

2. Key Components of java.awt

a. GUI Components

These are basic building blocks for creating GUI applications.

- **Container Classes:** Containers are components that can hold other components.
 - **Frame:** A top-level window with a title and borders.
 - **Panel:** A generic container for grouping components.
 - **Dialog:** A pop-up window used for messages or user input.
- **Control Components:** These components allow user interaction.
 - **Button:** A clickable button.
 - **Label:** A non-editable text field.
 - **TextField:** A single-line text input field.
 - **TextArea:** A multi-line text input field.
 - **Checkbox:** A toggle switch that can be selected or deselected.
 - **Choice:** A dropdown menu for selecting one option.
 - **List:** A list of selectable items.

b. Event Handling

The event-handling mechanism in `java.awt` enables interaction with GUI components.

- **Event Classes:**
 - **ActionEvent:** Represents actions like button clicks.
 - **MouseEvent:** Represents mouse actions like clicks and movement.
 - **KeyEvent:** Represents keyboard actions.
 - **WindowEvent:** Represents events related to windows (e.g., closing).
- **Event Listener Interfaces:**

- **ActionListener**: Handles action events.
 - **MouseListener**: Handles mouse events.
 - **KeyListener**: Handles keyboard events.
- **Adapter Classes**: Simplify event handling by providing default implementations.
 - **MouseAdapter**: For mouse events.
 - **WindowAdapter**: For window events.

c. Layout Managers

Control how components are arranged in a container.

- **FlowLayout**: Arranges components in a row, wrapping to the next line if needed.
- **BorderLayout**: Divides the container into five regions: NORTH, SOUTH, EAST, WEST, CENTER.
- **GridLayout**: Arranges components in a grid of rows and columns.
- **CardLayout**: Stacks components like a deck of cards, showing one at a time.

d. Graphics and Drawing

Provides tools for custom drawing and rendering.

- **Graphics**:
 - Class for rendering shapes, text, and images.
 - Methods: `drawLine()`, `drawRect()`, `drawString()`, `drawImage()`.
- **Color**:
 - Represents colors for GUI components or drawing.
 - Example: `Color.RED`, `Color(255, 255, 0)` for yellow.
- **Font**:
 - Represents text fonts.
 - Example: `new Font("Arial", Font.BOLD, 14)`.

e. Image Handling

Support for loading and displaying images.

- **Image**: Represents graphical images.
- **Toolkit**: Provides methods to load images and get system information. Example: `Toolkit.getDefaultToolkit().getImage("image.jpg")`.

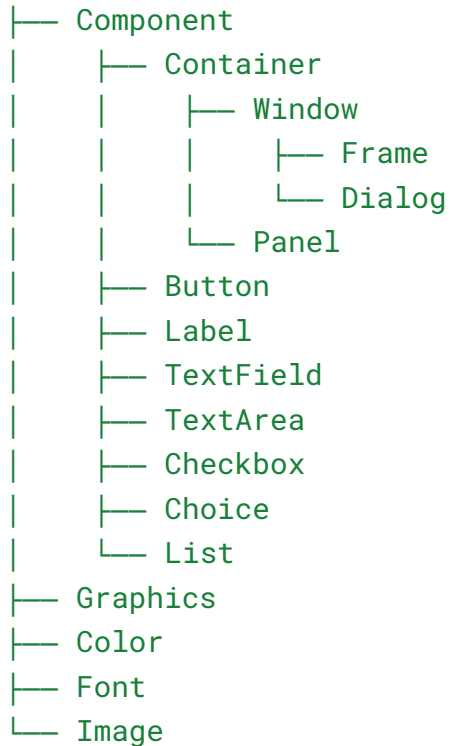
f. Clipboard and Data Transfer

- **Clipboard**: Manages data transfer between applications using cut, copy, and paste.
- **Transferable**: Interface for transferring data between clipboard and applications.

3. Class Hierarchy

Here's a simplified hierarchy of important classes in `java.awt`:

Object



4. Key Methods

Here are some commonly used methods in the `java.awt` package:

Creating Components:

```
Button button = new Button("Click Me");
Frame frame = new Frame("My Frame");
```

Adding Components to a Container:

```
frame.add(button);
```

Setting Layout:

```
frame.setLayout(new FlowLayout());
```

Event Handling:

```
button.addActionListener(new ActionListener() {  
    @Override  
    public void actionPerformed(ActionEvent e) {  
        System.out.println("Button clicked!");  
    }  
});
```

Custom Drawing:

java

Copy code

```
public void paint(Graphics g) {  
    g.setColor(Color.RED);  
    g.drawRect(50, 50, 100, 100);  
    g.drawString("Hello, AWT!", 60, 80);  
}
```

5. Example Program

Here's a complete example that demonstrates a simple `java.awt` application:

```
import java.awt.*;  
import java.awt.event.*;  
  
public class AWTEExample {  
    public static void main(String[] args) {  
        // Create a Frame  
        Frame frame = new Frame("AWT Example");  
        frame.setSize(400, 300);  
        frame.setLayout(new FlowLayout());  
  
        // Create a Button  
        Button button = new Button("Click Me");  
        frame.add(button);  
  
        // Add an ActionListener to the Button  
        button.addActionListener(new ActionListener() {  
            public void actionPerformed(ActionEvent e) {  
                System.out.println("Button clicked!");  
            }  
        })  
    }  
}
```

```
});

// Close the Frame on Window Close
frame.addWindowListener(new WindowAdapter() {
    public void windowClosing(WindowEvent e) {
        frame.dispose();
    }
});

// Make the Frame visible
frame.setVisible(true);
}
}
```

6. Limitations of java.awt

- AWT components are **heavyweight**, meaning they rely on the platform's native GUI components.
- Limited flexibility and customization compared to Swing.
- Swing is preferred for modern GUI development, but AWT is still used for its simplicity and as a foundation for Swing.

Two Event Handling Mechanisms in Java

Event handling in Java has evolved over time to become more robust and modular. There are two main mechanisms for handling events:

1. **Old Mechanism:** Based on **Action Commands** in early AWT.
2. **Modern Mechanism:** The **Delegation Event Model**, introduced in Java 1.1.

1. Old Mechanism: Action Commands (AWT Events)

Overview of Pre-1.1 Event Handling

Before Java 1.1, event handling was based on a centralized approach, where a single method, `handleEvent()`, was used to handle all events. This mechanism was part of the AWT (Abstract Window Toolkit).

- **How it Worked:**
 1. The `handleEvent()` method in the `Component` class was overridden to process events.
 2. Events were passed to this method, and the program had to identify the type of event manually.
- **Key Features:**
 1. Centralized event handling in one method.
 2. Event identification was done using `Event` class constants (e.g., `Event.ACTION_EVENT`).
- **Limitations:**
 1. **Lack of Modularity:**
 - All event handling code was in one place, leading to cluttered and hard-to-maintain code.
 2. **Inefficient:**
 - Developers had to manually check the type of event and its source.
 3. **Poor Scalability:**
 - As the application grew, this approach became unmanageable.

Example of Old Mechanism

Here's an example illustrating the limitations of pre-1.1 event handling:

```
import java.awt.*;
import java.awt.event.*;

public class OldEventHandlingExample extends Frame {
    public OldEventHandlingExample() {
        // Add a button to the frame
        Button button = new Button("Click Me");
        button.setBounds(50, 100, 80, 30);
        add(button);

        setSize(200, 200);
        setLayout(null);
        setVisible(true);
    }

    @Override
    public boolean handleEvent(Event e) {
        if (e.id == Event.ACTION_EVENT) { // Check if it's an action
event
            System.out.println("Button clicked!");
            return true; // Event handled
        }
        return super.handleEvent(e); // Pass unhandled events to the
superclass
    }

    public static void main(String[] args) {
        new OldEventHandlingExample();
    }
}
```

Drawbacks in the Example

- **Event Filtering:** Manually checking the event type (`e.id == Event.ACTION_EVENT`).
- **Centralized Code:** All logic is placed in `handleEvent()`.
- **No Separation:** Event source and handling logic are tightly coupled.

2. Modern Mechanism: Delegation Event Model

Introduction

The Delegation Event Model was introduced in Java 1.1 to overcome the limitations of the old mechanism. It separates the event generation (by the event source) and the event handling (by the event listener).

- **Core Idea:**
 - The event source generates events.
 - The event listener (or multiple listeners) is notified of these events.
 - This allows modular, scalable, and efficient event handling.

How It Works

1. **Event Source:**
 - The GUI component (e.g., `Button`) that generates an event when a user interacts with it.
 - Registers one or more listeners using methods like `addActionListener()`.
2. **Event Listener:**
 - An object that implements a specific listener interface (e.g., `ActionListener`).
 - Contains methods to handle the event (e.g., `actionPerformed()` for `ActionEvent`).
3. **Event Object:**
 - Encapsulates details about the event, such as the type of event, the source, and any additional data.
4. **Notification Process:**
 - When an event occurs, the source notifies all registered listeners by calling their methods with the event object.

Advantages

- **Separation of Concerns:**
 - The event-handling logic is decoupled from the event source.
- **Modularity:**
 - Each listener handles specific events, making the code more organized and maintainable.
- **Scalability:**
 - Multiple listeners can be registered to a single source, or a single listener can handle events from multiple sources.

Example of Modern Mechanism

Here's an example using the Delegation Event Model:

```
import java.awt.*;
import java.awt.event.*;

public class ModernEventHandlingExample extends Frame {
    public ModernEventHandlingExample() {
        // Create a button
        Button button = new Button("Click Me");
        button.setBounds(50, 100, 80, 30);

        // Add ActionListener to the button
        button.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                System.out.println("Button clicked!");
            }
        });

        // Add the button to the frame
        add(button);

        setSize(200, 200);
        setLayout(null);
        setVisible(true);

        // Close the frame on window close
        addWindowListener(new WindowAdapter() {
            @Override
            public void windowClosing(WindowEvent e) {
                dispose();
            }
        });
    }

    public static void main(String[] args) {
```

```
        new ModernEventHandlingExample();  
    }  
}
```

Features of Modern Mechanism in the Example

1. **Event Source:**
 - The `Button` is the event source, generating `ActionEvent`.
 2. **Event Listener:**
 - The `ActionListener` is implemented anonymously to handle the button click.
 3. **Event Object:**
 - The `ActionEvent` object contains details about the event.
-

Comparison of Old and Modern Mechanisms

Feature	Old Mechanism	Modern Mechanism
Introduction	Pre-Java 1.1	Introduced in Java 1.1
Event Filtering	Manual filtering in <code>handleEvent()</code>	Automatic delegation to listeners
Modularity	Poor	Excellent
Scalability	Poor	High
Ease of Use	Complex	Simple and organized

Conclusion

The modern Delegation Event Model is far superior to the old mechanism. By separating the event source and listener, it promotes cleaner, more maintainable, and modular code. It is the standard way to handle events in Java GUI programming.

The Delegation Event Model in Java

The **Delegation Event Model** is a core part of Java's event-handling mechanism, introduced in Java 1.1. It separates event generation from event handling, providing a robust and modular way to handle user interactions in GUI applications.

Key Components of the Delegation Event Model

1. **Event Source:**
 - The object that generates an event when a user interacts with it.
 - Examples: `JButton`, `TextField`, `ComboBox`, etc.
 - Responsibilities:
 - Registers listeners that want to respond to its events.
 - Notifies registered listeners when an event occurs.
2. **Event Listener:**
 - An object that listens for and processes events.
 - Implements specific listener interfaces (e.g., `ActionListener`, `MouseListener`).
 - Responsibilities:
 - Defines methods to respond to specific types of events.
 - Registers itself with one or more event sources.
3. **Event Object:**
 - An object that encapsulates all details about an event.
 - Examples: `ActionEvent`, `MouseEvent`, `KeyEvent`, etc.
 - Responsibilities:
 - Provides information such as the event type, source, and any additional

Workflow of the Delegation Event Model

The Delegation Event Model operates as follows:

1. **Event Generation:**
 - A user interacts with a GUI component (e.g., clicks a button or types in a text field).
 - The GUI component acts as the **event source** and generates an event.
2. **Event Notification:**
 - The event is sent to all registered listeners for that event type.
 - Listeners are objects that have registered themselves with the event source.
3. **Event Handling:**
 - Each listener executes the appropriate method (defined in the listener interface) to handle the event.

This workflow ensures that the event-handling logic is cleanly separated from the GUI components generating the events.

Example Implementation

Let's look at a practical example where a button click event is handled using the Delegation Event Model.

```
import java.awt.event.*; // For event classes and interfaces
import javax.swing.*;    // For GUI components

public class DelegationEventModelExample {
    public static void main(String[] args) {
        // Create a JFrame (window)
        JFrame frame = new JFrame("Delegation Event Model Example");
        frame.setSize(300, 200);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        // Create a JButton (event source)
        JButton button = new JButton("Click Me");
        frame.add(button);

        // Add an ActionListener to the button (event listener)
        button.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                // Handle the button click event
                System.out.println("Button was clicked!");
            }
        });

        // Make the frame visible
        frame.setVisible(true);
    }
}
```

Example

1. **Event Source:**
 - `JButton` (`button`) generates an `ActionEvent` when clicked.
2. **Event Listener:**
 - The anonymous class implementing `ActionListener` defines the `actionPerformed()` method, which responds to the button click.
3. **Event Object:**
 - The `ActionEvent` object contains details about the event, such as the source (`button`) and the time of the event.
4. **Workflow:**
 - The `button` generates an `ActionEvent` when clicked.
 - The `ActionEvent` is sent to the registered listener (the anonymous class).
 - The listener's `actionPerformed()` method is executed, printing a message to the console.

Common Event Classes and Listener Interfaces

Event Class	Event Listener Interface	Purpose
<code>ActionEvent</code>	<code>ActionListener</code>	Handles button clicks, menu selections, etc.
<code>MouseEvent</code>	<code>MouseListener</code> , <code>MouseMotionListener</code>	Handles mouse actions like clicks, drags, etc.
<code>KeyEvent</code>	<code>KeyListener</code>	Handles keyboard actions.
<code>FocusEvent</code>	<code>FocusListener</code>	Handles focus gain or loss of a component.
<code>WindowEvent</code>	<code>WindowListener</code>	Handles window-related actions.

Advantages of the Delegation Event Model

1. **Separation of Concerns:**
 - The event-handling logic is decoupled from the GUI component that generates the event.
2. **Modularity:**
 - Each listener handles specific events, making the code cleaner and easier to maintain.
3. **Scalability:**

- A single source can notify multiple listeners, or a single listener can handle events from multiple sources.
4. **Reusability:**
- Listeners can be reused for multiple sources if they implement common behavior.

Conclusion

The Delegation Event Model is the foundation of Java's modern event-handling system. By separating the event source, listener, and event object, it promotes clean, modular, and scalable code for GUI applications. This model is widely used in both AWT and Swing, making it essential for Java developers to understand and apply effectively.

Event Classes in Java

In Java, **event classes** represent the different types of events generated by user interactions with GUI components. Each event class encapsulates information about the event, such as its source, type, and any additional details relevant to the event type. These classes are part of the `java.awt.event` package and are used extensively in AWT and Swing-based GUI applications.

Common Event Classes and Their Use Cases

1. **ActionEvent:**

- Represents action events like button clicks, menu selections, etc.
- Common sources: `JButton`, `JMenuItem`, `JTextField`.
- **Key Method:**
 - `getActionCommand()`: Returns the command string associated with the event (e.g., the label of a button).

Example:

```
JButton button = new JButton("Submit");
button.addActionListener(e -> System.out.println("Command: " +
e.getActionCommand()));
```

2. **MouseEvent:**

- Represents mouse interactions like clicks, movement, dragging, and entering/exiting a component.
- Common sources: `JPanel`, `JButton`, etc.
- **Key Methods:**
 - `getButton()`: Returns which mouse button (left, middle, or right) was pressed.
 - `getX()` and `getY()`: Return the coordinates of the mouse pointer during the event.

Example:

```
frame.addMouseListener(new MouseAdapter() {
    public void mouseClicked(MouseEvent e) {
        System.out.println("Clicked at: " + e.getX() + ", " +
e.getY());
        System.out.println("Button: " + e.getButton());
    }
});
```

```
});
```

3. **KeyEvent:**

- Represents keyboard interactions like pressing, releasing, or typing a key.
- Common sources: `JTextField`, `JTextArea`, or any component with keyboard focus.
- **Key Methods:**
 - `getKeyCode()`: Returns the integer code of the key that was pressed.
 - `getKeyChar()`: Returns the character representation of the key.

Example:

```
frame.addKeyListener(new KeyAdapter() {  
    public void keyPressed(KeyEvent e) {  
        System.out.println("Key Pressed: " + e.getKeyChar() + " (" +  
e.getKeyCode() + ")");  
    }  
});
```

4. **WindowEvent:**

- Represents window-related interactions like opening, closing, or minimizing a window.
- Common sources: `JFrame`, `JDialog`.
- **Key Methods:**
 - `getWindow()`: Returns the `Window` object where the event occurred.

Example:

```
frame.addWindowListener(new WindowAdapter() {  
    public void windowClosing(WindowEvent e) {  
        System.out.println("Window is closing");  
    }  
});
```

5. **FocusEvent:**

- Represents focus gain or loss events on a component.
- Common sources: `JTextField`, `JButton`, etc.
- **Key Methods:**
 - `isTemporary()`: Indicates whether the focus change is temporary.

Example:

```
textField.addFocusListener(new FocusAdapter() {
    public void focusGained(FocusEvent e) {
        System.out.println("Focus gained on text field");
    }
});
```

Detailed Explanation of Important Methods

1. **ActionEvent:**

`getActionCommand()`: Retrieves a string identifying the action command. For example, in a `JButton`, this might return the button's label or a custom command set using `setActionCommand()`.

Example:

```
JButton button = new JButton("Save");
button.setActionCommand("SaveCommand");
button.addActionListener(e -> System.out.println("Action: " +
e.getActionCommand()));
```

2. **MouseEvent:**

`getButton()`: Returns an integer indicating which button (1 for left, 2 for middle, 3 for right) was pressed.

Example:

```
if (e.getButton() == MouseEvent.BUTTON1) {
    System.out.println("Left button clicked");
}
```

3. **KeyEvent:**

`getKeyCode()`: Returns the virtual keycode (an integer constant) for the key pressed (e.g., `KeyEvent.VK_ENTER`).

Example:

```
if (e.getKeyCode() == KeyEvent.VK_ENTER) {
    System.out.println("Enter key pressed");
}
```

`getKeyChar()`:

Returns the character corresponding to the key pressed (e.g., 'a', 'A', or '1').

Example:

```
System.out.println("Key typed: " + e.getKeyChar());
```

Example Program: Combining Event Classes

Below is an example program that demonstrates the usage of multiple event classes:

```
import java.awt.event.*;
import javax.swing.*;

public class EventClassExample {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Event Class Demo");
        frame.setSize(400, 300);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setLayout(null);

        JButton button = new JButton("Click Me");
        button.setBounds(50, 50, 100, 50);
        frame.add(button);

        JTextField textField = new JTextField();
        textField.setBounds(50, 150, 200, 30);
        frame.add(textField);

        // ActionEvent
        button.addActionListener(e -> System.out.println("Button
clicked!"));

        // KeyEvent
        textField.addKeyListener(new KeyAdapter() {
            public void keyTyped(KeyEvent e) {
                System.out.println("Key typed: " + e.getKeyChar());
            }
        });
    }
}
```

```
// MouseEvent
frame.addMouseListener(new MouseAdapter() {
    public void mouseClicked(MouseEvent e) {
        System.out.println("Mouse clicked at: " + e.getX() +
            ", " + e.getY());
    }
});

frame.setVisible(true);
}
```

Conclusion

Event classes in Java provide a powerful way to handle diverse user interactions, enabling developers to create interactive and responsive GUI applications. Understanding these classes and their methods is essential for effectively implementing event-driven programming in Java.

Sources of Events

Event sources are GUI components that generate events in response to user interactions. These components serve as the starting point of the event-handling process.

Examples of Event Sources

1. Buttons (e.g., **JButton**):

- Clicking a button generates an **ActionEvent**.

Example:

```
JButton button = new JButton("Click Me");  
button.addActionListener(e -> System.out.println("Button clicked!"));
```

2. Text Fields (e.g., **JTextField**):

- Pressing Enter generates an **ActionEvent**.

Example:

```
JTextField textField = new JTextField();  
textField.addActionListener(e -> System.out.println("Text Entered: " +  
textField.getText()));
```

3. Sliders (e.g., **JSlider**):

- Adjusting the slider generates a **ChangeEvent**.

Example:

```
JSlider slider = new JSlider();  
slider.addChangeListener(e -> System.out.println("Slider Value: " +  
slider.getValue()));
```

4. Windows (e.g., **JFrame**, **JDialog**):

- Closing or opening a window generates a **WindowEvent**.

Example:

```
frame.addWindowListener(new WindowAdapter() {  
    public void windowClosing(WindowEvent e) {  
        System.out.println("Window is closing.");  
    }  
});
```

Demonstration: Button Click Generating an Event

Here's a simple example of a button click generating an event:

```
import javax.swing.*;  
import java.awt.event.*;  
  
public class EventSourceDemo {  
    public static void main(String[] args) {  
        JFrame frame = new JFrame("Event Source Example");  
        frame.setSize(300, 200);  
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
  
        JButton button = new JButton("Click Me");  
        button.addActionListener(e -> System.out.println("Button  
clicked!"));  
  
        frame.add(button);  
        frame.setVisible(true);  
    }  
}
```

6. Event Listener Interfaces

Event listener interfaces define the methods that respond to specific types of events. When a component generates an event, the corresponding listener's methods are executed to handle it.

Key Listener Interfaces

1. ActionListener:

- **Purpose:** Handles action events like button clicks or pressing Enter in a text field.
- **Method:** `actionPerformed(ActionEvent e)`

Example:

```
JButton button = new JButton("Submit");
button.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        System.out.println("Button clicked!");
    }
});
```

2. **MouseListener:**

- **Purpose:** Handles mouse events like clicks, entering, and exiting a component.
- **Methods:**
 - mouseClicked(MouseEvent e)
 - mouseEntered(MouseEvent e)
 - mouseExited(MouseEvent e)
 - mousePressed(MouseEvent e)
 - mouseReleased(MouseEvent e)

Example:

```
frame.addMouseListener(new MouseAdapter() {
    public void mouseClicked(MouseEvent e) {
        System.out.println("Mouse clicked!");
    }
});
```

3. **KeyListener:**

- **Purpose:** Handles keyboard events like pressing, releasing, or typing keys.
- **Methods:**
 - keyPressed(KeyEvent e)
 - keyReleased(KeyEvent e)
 - keyTyped(KeyEvent e)

Example:

```
textField.addKeyListener(new KeyAdapter() {
    public void keyPressed(KeyEvent e) {
        System.out.println("Key Pressed: " + e.getKeyChar());
    }
});
```

Putting It All Together

Here's a complete program that demonstrates the use of event classes, sources, and listener interfaces:

```
import javax.swing.*;
import java.awt.event.*;

public class EventListenerDemo {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Event Listener Demo");
        frame.setSize(400, 300);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        JButton button = new JButton("Click Me");
        button.setBounds(50, 50, 100, 50);
        button.addActionListener(e -> System.out.println("Button
clicked!"));

        JTextField textField = new JTextField();
        textField.setBounds(50, 150, 200, 30);
        textField.addKeyListener(new KeyAdapter() {
            public void keyPressed(KeyEvent e) {
                System.out.println("Key Pressed: " + e.getKeyChar());
            }
        });

        frame.add(button);
        frame.add(textField);
        frame.setLayout(null);
        frame.setVisible(true);
    }
}
```

This program showcases event classes ([ActionEvent](#), [KeyEvent](#)), sources ([JButton](#), [JTextField](#)), and listeners ([ActionListener](#), [KeyListener](#)) working together in a typical Java GUI application.

Adapter Classes

Purpose of Adapter Classes

In Java, many listener interfaces contain multiple methods. For example, the `MouseListener` interface has five methods. If a developer wants to handle only a specific event (e.g., mouse clicks), they are still required to provide implementations for all methods in the interface, even if most are left empty.

Adapter classes solve this issue by providing default (empty) implementations for all the methods in a listener interface. Developers can then create a subclass of the adapter and override only the methods they need, simplifying event handling.

Examples of Adapter Classes

1. `MouseAdapter`:

- Corresponds to the `MouseListener` interface, which has multiple methods like `mouseClicked`, `mouseEntered`, `mouseExited`, etc.
- Instead of implementing all methods, you can extend `MouseAdapter` and override only the required methods.

Example:

```
import javax.swing.*;
import java.awt.event.*;

public class MouseAdapterDemo {
    public static void main(String[] args) {
        JFrame frame = new JFrame("MouseAdapter Example");
        frame.setSize(300, 200);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        JLabel label = new JLabel("Hover or Click");
        label.setBounds(100, 70, 100, 30);
        frame.add(label);

        label.addMouseListener(new MouseAdapter() {
            @Override
            public void mouseClicked(MouseEvent e) {
                System.out.println("Mouse clicked at: " + e.getX() +
                    ", " + e.getY());
            }
        });
    }
}
```

```

    }

    @Override
    public void mouseEntered(MouseEvent e) {
        System.out.println("Mouse entered the label.");
    }
});

frame.setLayout(null);
frame.setVisible(true);
}
}

```

2. **KeyAdapter:**

- Corresponds to the `KeyListener` interface, which has methods like `keyPressed`, `keyReleased`, and `keyTyped`.
- Extend `KeyAdapter` to override only the required methods.

Example:

```

import javax.swing.*;
import java.awt.event.*;

public class KeyAdapterDemo {
    public static void main(String[] args) {
        JFrame frame = new JFrame("KeyAdapter Example");
        frame.setSize(300, 200);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        JTextField textField = new JTextField();
        textField.setBounds(50, 50, 200, 30);
        frame.add(textField);

        textField.addKeyListener(new KeyAdapter() {
            @Override
            public void keyTyped(KeyEvent e) {
                System.out.println("Key typed: " + e.getKeyChar());
            }
        });
    }
}

```

```
        frame.setLayout(null);  
        frame.setVisible(true);  
    }  
}
```

Usage of Adapter Classes

1. Extend the appropriate adapter class (`MouseAdapter`, `KeyAdapter`, etc.).
2. Override only the methods relevant to your application.
3. Attach the adapter object to the component using the appropriate `addListener` method.

Benefits:

- Reduces boilerplate code.
- Improves readability by focusing only on relevant methods.

Inner Classes in Event Handling

What are Inner Classes?

Inner classes are classes defined within the body of another class. In event handling, they are often used to encapsulate the event-handling logic close to the source of the event.

Advantages of Inner Classes in Event Handling

1. **Encapsulation:**
 - Keeps event-handling code closely associated with the GUI component.
 - Reduces clutter in the main class by organizing related code together.
2. **Direct Access:**
 - Inner classes have access to all members (fields and methods) of the enclosing class, even private ones.
 - Simplifies the interaction between the event-handling code and the enclosing class.
3. **Simpler Syntax:**
 - Avoids the need for explicit references to the enclosing class.

Example of Using an Inner Class

```
import javax.swing.*;
import java.awt.event.*;

public class InnerClassDemo {
    private JButton button;
    private JLabel label;

    public InnerClassDemo() {
        JFrame frame = new JFrame("Inner Class Example");
        frame.setSize(300, 200);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setLayout(null);

        button = new JButton("Click Me");
        button.setBounds(100, 50, 100, 30);
        frame.add(button);

        label = new JLabel("Waiting...");
        label.setBounds(100, 100, 100, 30);
```

```

        frame.add(label);

        button.addActionListener(new ButtonClickListener()); // Use of
inner class

        frame.setVisible(true);
    }

    // Inner class for handling button clicks
    private class ButtonClickListener implements ActionListener {
        @Override
        public void actionPerformed(ActionEvent e) {
            label.setText("Button Clicked!");
        }
    }

    public static void main(String[] args) {
        new InnerClassDemo();
    }
}

```

When to Use Inner Classes

- When event-handling logic is specific to one GUI component.
- When the handler needs direct access to the enclosing class's variables or methods.

Key Comparison: Adapter Classes vs. Inner Classes

Feature	Adapter Classes	Inner Classes
Purpose	Simplify implementation of listener interfaces by overriding only needed methods.	Organize event-handling logic within the main class.
Usage	Extend the adapter class and override methods.	Define an inner class and implement the listener interface.
Access to Enclosing Class	Not directly accessible.	Direct access to all members of the enclosing class.

Best Use Case	When a listener interface has many methods, but only a few are needed.	When event-handling logic is tightly coupled with a single component.
----------------------	--	---

Both techniques simplify Java event handling and improve code readability while keeping the logic modular and maintainable.